

pgBackRest Lab Guide

PostgreSQL 18 + Patroni/Spilo + Kubernetes

Full Backup • Incremental Backup • Differential Backup • Restore • PITR

Namespace	postgres-ha
Primary pod	acid-postgres-ha-0
Replica pods	acid-postgres-ha-1, acid-postgres-ha-2
PostgreSQL	18.3
pgBackRest	2.59.1
Confirmed PGDATA	/home/postgres/pgdata/pgroot/data

Lab safety - Do not restore directly over the running Patroni leader. This guide restores into a separate directory for validation. For production HA, use a shared repository such as S3, NFS or an RWX PVC.

Contents

1. 1. Why the earlier commands failed
2. 2. Lab architecture and paths
3. 3. Step 1 - Set shell variables
4. 4. Step 2 - Verify leader and replicas
5. 5. Step 3 - Verify PGDATA and pgBackRest
6. 6. Step 4 - Verify persistent storage
7. 7. Step 5 - Create repository
8. 8. Step 6 - Create pgBackRest configuration
9. 9. Step 7 - Create and check stanza
10. 10. Step 8 - Configure WAL archiving
11. 11. Lab Exercise 1 - Full backup
12. 12. Lab Exercise 2 - Incremental backup
13. 13. Lab Exercise 3 - Differential backup
14. 14. Lab Exercise 4 - Restore
15. 15. Lab Exercise 5 - Point-in-Time Recovery
16. 16. Troubleshooting
17. 17. Quick command reference

1. Why the earlier commands failed

```

ps@ubuntu24:~$ kubectl exc -n postgres-ha acid-postgres-ha-0 -- printenv PGDATA
error: unknown command "exc" for "kubectl"

Did you mean this?
    exec
ps@ubuntu24:~$ kubectl exec -n postgres-ha acid-postgres-ha-0 -- printenv PGDATA
/home/postgres/pgdata/pgroot/data
ps@ubuntu24:~$ cat > /etc/pgbackrest/pgbackrest.conf << 'EOF'
> [global]
> rep1-path=/var/lib/pgbackrest
> rep1-retention-full=2
> rep1-retention-diff=3
> start-fast=y
> process-max=2
> log-level-console=info
> [acid-postgres-ha]
> pg1-path=/home/postgres/pgdata/pgroot/data
> EOF
bash: /etc/pgbackrest/pgbackrest.conf: No such file or directory
ps@ubuntu24:~$

```

Observed issue: host-level file creation was attempted instead of creating the configuration inside the PostgreSQL pod.

Issue 1 - The first kubectl command had a typo: "exc" instead of "exec". The corrected kubectl exec command worked.

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- printenv PGDATA
```

Issue 2 - The confirmed PGDATA path is /home/postgres/pgdata/pgroot/data. Use this exact value for pg1-path.

Issue 3 - The command cat > /etc/pgbackrest/pgbackrest.conf was run at the Ubuntu host prompt, so Bash tried to create the file on the host. The configuration needs to be created inside the PostgreSQL pod, or supplied explicitly with --config.

Issue 4 - Earlier /var/lib/pgbackrest appeared on the container overlay filesystem. This guide uses a lab repository under /home/postgres/pgdata so it is placed under the PostgreSQL mounted storage path.

2. Lab architecture and paths

```

Ubuntu host shell
|
+-- kubectl exec --> acid-postgres-ha-0    PRIMARY
|
|          PGDATA: /home/postgres/pgdata/pgroot/data
|          Config: /home/postgres/pgdata/pgbackrest.conf
|          Repo:   /home/postgres/pgdata/pgbackrest-repo
|
+-- acid-postgres-ha-1    REPLICA
+-- acid-postgres-ha-2    REPLICA

```

3. Step 1 - Set shell variables

```

export NS=postgres-ha
export PRIMARY=acid-postgres-ha-0
export STANZA=acid-postgres-ha
export PGDATA=/home/postgres/pgdata/pgroot/data
export PGBR_CFG=/home/postgres/pgdata/pgbackrest.conf
export PGBR_REPO=/home/postgres/pgdata/pgbackrest-repo

echo $NS
echo $PRIMARY
echo $STANZA
echo $PGDATA
echo $PGBR_CFG
echo $PGBR_REPO

```

Expected: Variables display the values shown above.

4. Step 2 - Verify leader and replicas

```
kubectl get pods -n $NS -L spilo-role
```

Expected: acid-postgres-ha-0 = master; acid-postgres-ha-1 and -2 = replica.

Note - Pooler pods may also carry spilo-role labels. Use only acid-postgres-ha-0/1/2 to identify database roles.

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -Atc "SELECT CASE WHEN
pg_is_in_recovery() THEN 'REPLICA' ELSE 'PRIMARY' END;"
```

Expected: PRIMARY

5. Step 3 - Verify PGDATA and pgBackRest

```
kubectl exec -n $NS $PRIMARY -- printenv PGDATA
```

Expected: /home/postgres/pgdata/pgroot/data

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -Atc "SHOW data_directory;"
```

Expected: /home/postgres/pgdata/pgroot/data

```
kubectl exec -n $NS $PRIMARY -- pgbackrest version
```

Expected: pgBackRest 2.59.1

6. Step 4 - Verify persistent storage

```
kubectl exec -n $NS $PRIMARY -- df -hT /home/postgres/pgdata
```

Expected: The filesystem should correspond to the PostgreSQL mounted volume, not just the pod root overlay.

```
kubectl exec -n $NS $PRIMARY -- mount | grep /home/postgres/pgdata
kubectl describe pod -n $NS $PRIMARY | sed -n '/Mounts:/,/Conditions:/p'
```

Production note - A repository stored under a pod-specific PVC is appropriate only for a controlled lab. In production HA, the backup repository must remain available if the primary pod fails or leadership moves.

7. Step 5 - Create pgBackRest repository

```
kubectl exec -n $NS $PRIMARY -- mkdir -p $PGBR_REPO
kubectl exec -n $NS $PRIMARY -- ls -ld $PGBR_REPO
kubectl exec -n $NS $PRIMARY -- sh -c 'touch /home/postgres/pgdata/pgbackrest-repo/.write-test && echo "WRITE OK" && rm /home/postgres/pgdata/pgbackrest-repo/.write-test'
```

Expected: WRITE OK

8. Step 6 - Create pgBackRest configuration

Important - Run the following complete block from the Ubuntu host. `kubectl exec -i` passes the configuration into the pod through standard input.

```
kubectl exec -i -n $NS $PRIMARY -- \
sh -c 'cat > /home/postgres/pgdata/pgbackrest.conf' <<'EOF'
[global]
repo1-path=/home/postgres/pgdata/pgbackrest-repo
repo1-retention-full=2
repo1-retention-diff=3
start-fast=y
process-max=2
log-level-console=info

[acid-postgres-ha]
pg1-path=/home/postgres/pgdata/pgroot/data
EOF
kubectl exec -n $NS $PRIMARY -- cat $PGBR_CFG
```

Expected: The configuration is displayed exactly as entered.

```
kubectl exec -n $NS $PRIMARY -- ls -l $PGBR_CFG
```

Expected: The file exists inside the PostgreSQL pod.

9. Step 7 - Create and check stanza

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA stanza-create
```

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA info
```

Expected: The stanza is recognized.

10. Step 8 - Configure WAL archiving

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SHOW archive_mode;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SHOW archive_command;"
```

Do not overwrite blindly - If archive_command already uses WAL-G, Barman or another archive solution, do not replace it on a production cluster. The next ALTER SYSTEM command is intended for this disposable lab.

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "ALTER SYSTEM SET archive_command = 'pgbackrest --config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha archive-push %p';"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT pg_reload_conf();"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SHOW archive_command;"
```

Expected: archive_command contains the pgBackRest archive-push command.

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT pg_switch_wal();"
sleep 5
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT archived_count, failed_count, last_archived_wal, last_failed_wal FROM pg_stat_archiver;"
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA check
```

Expected: check command end: completed successfully

Prepare sample data

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "CREATE TABLE IF NOT EXISTS pgbackrest_lab
(id integer PRIMARY KEY,
description text,
created_at timestamptz DEFAULT clock_timestamp());"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO pgbackrest_lab(id,description)
VALUES (1,'Before full backup'),(2,'Full backup baseline')
ON CONFLICT (id) DO NOTHING;"
```

11. Lab Exercise 1 - Full backup

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA --type=full backup
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA info
```

Expected: A backup entry of type full appears.

- Full backup contains all database files required for the backup set.
- It becomes the base for later incremental and differential backups.

12. Lab Exercise 2 - Incremental backup

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description)
VALUES (3,'Created after full backup'),(4,'Included in incremental backup');"
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA --type=incr backup
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA info
```

Expected: A new incremental backup appears after the full backup.

Concept - Incremental backup captures changes since the most recent backup, whether that prior backup was full, differential or incremental.

13. Lab Exercise 3 - Differential backup

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description)
VALUES (5,'Created before differential backup'),(6,'Differential backup test');"
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA --type=diff backup
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA info
```

Expected: A differential backup appears.

Concept - Differential backup captures changes since the most recent full backup.

14. Lab Exercise 4 - Restore safely

Safety - Do not restore over the live Patroni PGDATA. Restore into a separate directory for inspection or use a separate recovery pod.

```
kubectl exec -n $NS $PRIMARY -- sh -c 'rm -rf /home/postgres/pgdata/restore-test && mkdir -p
/home/postgres/pgdata/restore-test'
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA
--pg1-path=/home/postgres/pgdata/restore-test restore
kubectl exec -n $NS $PRIMARY -- ls -lh /home/postgres/pgdata/restore-test
```

Expected: Restored PostgreSQL data files are present.

```
kubectl exec -n $NS $PRIMARY -- cat /home/postgres/pgdata/restore-test/PG_VERSION
```

Expected: 18

```
kubectl exec -n $NS $PRIMARY -- ls -l /home/postgres/pgdata/restore-test/global/pg_control
```

Expected: pg_control exists.

15. Lab Exercise 5 - Point-in-Time Recovery (PITR)

PITR principle - PITR restores a physical backup and replays archived WAL only up to the specified target time.

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description) VALUES (100,'KEEP THIS ROW FOR PITR');"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT clock_timestamp();"
```

Record this time - Write down the timestamp. It becomes the PITR target. Wait a few seconds before simulating the accidental delete.

```
sleep 5
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "DELETE FROM pgbackrest_lab WHERE
id=100;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT pg_switch_wal();"
sleep 5
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT
archived_count,failed_count,last_archived_wal FROM pg_stat_archiver;"
kubectl exec -n $NS $PRIMARY -- sh -c 'rm -rf /home/postgres/pgdata/pitr-test && mkdir -p
/home/postgres/pgdata/pitr-test'
```

Replace the timestamp - The following command contains an example target time. Replace it with the timestamp you recorded before the DELETE.

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --config=$PGBR_CFG --stanza=$STANZA
--pg1-path=/home/postgres/pgdata/pitr-test --type=time --target="2026-09-25 06:30:15+00" --
target-action=promote restore
kubectl exec -n $NS $PRIMARY -- ls -l /home/postgres/pgdata/pitr-test
```

Expected: Restored files are present. Start this data directory only in a separate recovery instance/pod to validate recovered data.

16. Troubleshooting

Symptom	Correction
kubectl: unknown command "exc"	Use kubectl exec, not kubectl exc.
/etc/pgbackrest/pgbackrest.conf: No such file or directory	You created the file on the Ubuntu host. Use kubectl exec -i to create it inside the pod, or create the directory inside the pod first.
touch: missing file operand	The command was split after touch. Keep the file argument on the same shell command, preferably inside sh -c with quotes.
pgBackRest cannot find stanza	Run stanza-create using the same --config and --stanza values used for backup.
archive-push/check fails	Check archive_mode, archive_command, repository

	write permission, pgBackRest config path and pg1-path.
repository disappears after pod replacement	The repository is not on persistent/shared storage. Use S3, NFS or RWX PVC for production.
restore attempted on active Patroni leader	Stop. Restore to a separate path/pod for validation. A production Patroni recovery procedure requires coordinated cluster management.

17. Quick command reference

Task	Command
Leader/replicas	<code>kubectl get pods -n \$NS -L spilo-role</code>
PGDATA	<code>kubectl exec -n \$NS \$PRIMARY -- printenv PGDATA</code>
Database data directory	<code>kubectl exec -n \$NS \$PRIMARY -- psql -U postgres -d postgres -Atc "SHOW data_directory;"</code>
pgBackRest version	<code>kubectl exec -n \$NS \$PRIMARY -- pgbackrest version</code>
Stanza create	<code>pgbackrest --config=\$PGBR_CFG --stanza=\$STANZA stanza-create</code>
Check	<code>pgbackrest --config=\$PGBR_CFG --stanza=\$STANZA check</code>
Full	<code>pgbackrest --config=\$PGBR_CFG --stanza=\$STANZA --type=full backup</code>
Incremental	<code>pgbackrest --config=\$PGBR_CFG --stanza=\$STANZA --type=incr backup</code>
Differential	<code>pgbackrest --config=\$PGBR_CFG --stanza=\$STANZA --type=diff backup</code>
Backup info	<code>pgbackrest --config=\$PGBR_CFG --stanza=\$STANZA info</code>

Completion criteria - The lab is complete when stanza-check succeeds, full/incremental/differential backups appear in pgbackrest info, a restore directory contains a valid PostgreSQL 18 cluster, and WAL archiving is healthy with no failed archive operations.